ASSIGNMENT 1 – TASK 1: Understand the Problem and Identify the Objects

Bloom's Level: K4 – Analyse | Marks: 4

The system manages a multi-floor campus car park for students, staff and visitors. It supports two-wheelers, cars and EVs; EV charging is billed separately. On entry, the system allocates a suitable free slot, creates a unique ticket and records entry time. On exit, it calculates the fee, closes the ticket and releases the slot.

Inputs and outputs

Inputs	Outputs
Registration number, vehicle category, user type, monthly-pass validity	Allocated floor and slot, or a full/incompatible-slot message
Facility floor capacities and slot features	Unique ticket number and entry timestamp
Entry/exit time, pricing rule, EV charging units	Parking fee, separate charging fee, released slot and receipt

Candidate classes and why they are needed

Class / interface	Responsibility
Vehicle (abstract)	Common registration data and common fee contract; prevents direct creation of a generic vehicle.
TwoWheeler, Car, ElectricCar	Represent category-specific behaviour/rates; new categories can be added as subclasses.
ParkingSlot	Stores floor/slot identity, slot type and occupancy; enforces one vehicle per slot.
ParkingFacility	Owns floors/slots and allocates/releases spaces; keeps the search policy in one place.
ParkingTicket	Stores unique ticket number, vehicle, slot, entry/exit time and ticket state.
TicketNumberGenerator	Generates non-reused ticket numbers; centralises uniqueness instead of scattering counters.
PricingStrategy	Calculates parking fee separately from vehicle identity, allowing new fee rules without rewriting vehicle classes.
Chargeable	Capability interface for vehicles/services that can charge; charging is billed separately.
MonthlyPass / User	Represents user type and pass validity; only eligible students/staff with a valid pass receive free parking.

Nouns, verbs and methods

Nouns suggest candidate classes/fields: vehicle, floor, slot, ticket, fee, pass, user and facility. Verbs suggest methods: enter/allocate(), issueTicket(), recordEntry(), calculateFee(), chargingFee(), exit(), release(), validatePass() and closeTicket().

Business rules and assumptions

- A slot can contain at most one vehicle; allocation must be atomic so two entry requests cannot claim the same slot.

- Ticket numbers are unique and never reused during the system lifetime. A ticket has one entry and at most one successful exit.
- Only a compatible free slot may be allocated. EV charging is charged separately and only in a charging-enabled slot when required.
- Staff/students with a valid monthly pass pay zero parking fee; visitors pay the configured fee. Any charging fee remains separate unless the policy explicitly waives it.
- Exit time must not precede entry time. Invalid registration numbers, unknown categories and invalid slot indexes are rejected.
- Assumption: the university supplies the real rates, duration slabs, rounding policy, pass validity rules and definition of charging units. Examples in this document are clearly illustrative.

Alternative: objects versus parallel arrays

With objects, Vehicle v stores related fields and methods together. Searching a list of n vehicles by registration is O(n) unless a Map index is maintained; adding a field usually means changing the Vehicle class and its constructors/accessors. With parallel arrays such as regNo[], type[], entryTime[], every vehicle's data must remain at the same index. A search is also O(n), and adding an attribute requires another array plus careful synchronized updates. Both use O(n) total space (O(1) per vehicle record); objects are safer and easier to extend because related data stays together.